

E-Commerce Sales Data Analysis Using Python

A Data Analytics Project Using Pandas, NumPy and Matplotlib

Project Type: Data Analytics

Domain: E-Commerce / Retail

Programming Language: Python

Data Processing: Pandas, NumPy

Data Visualization: Matplotlib

Dataset: E-Commerce Transaction Dataset

Analysis Period: One Year (2025 -Jan to 2025 – Dec)

Perumalla Elisha Babu Python Developer(Full-stack, Data Analyst)

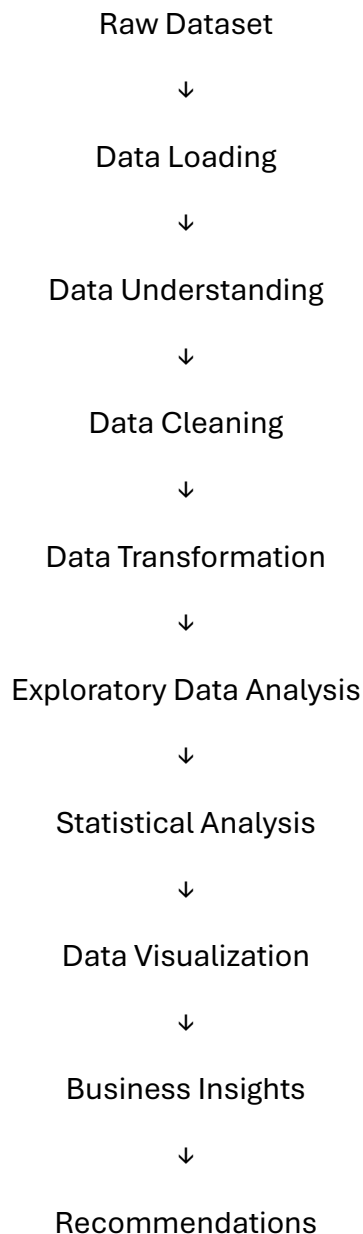
2. PROJECT OVERVIEW

2.1 Introduction

The E-Commerce Sales Data Analysis project focuses on analyzing one year of e-commerce transaction data to understand sales performance, customer purchasing behavior, product performance, regional sales distribution, payment preferences, discount effectiveness, and unusual transactions. The project uses Python-based data analytics techniques to transform raw transactional data into meaningful business insights. The analysis is performed using:

- Python
- Pandas
- NumPy
- Matplotlib

The project follows a complete data analytics workflow:



3. PROJECT OBJECTIVES

The primary objective of this project is to analyze e-commerce transaction data and identify meaningful patterns and trends.

Specific objectives

1. Calculate total revenue generated.
2. Calculate the total number of orders.
3. Calculate Average Order Value.
4. Analyze monthly revenue trends.
5. Identify the top 10 best-performing products.
6. Analyze sales performance by category.
7. Analyze sales performance by region.
8. Identify the most popular payment method.
9. Identify the highest and lowest sales months.
10. Analyze the relationship between discounts and sales.
11. Detect unusually high and low-value transactions.
12. Generate meaningful visualizations.
13. Extract actionable business insights.
14. Provide recommendations based on the analysis

4. BUSINESS PROBLEM STATEMENT

An e-commerce company generates a large amount of transaction data every day. However, raw transaction data alone does not provide a clear understanding of business performance.

- The company needs to answer questions such as:
- How much revenue did the business generate?
- Which products generate the highest sales?
- Which categories perform best?
- Which regions contribute the most revenue?
- Which months have the highest and lowest sales?
- Which payment method is most commonly used?
- Does offering higher discounts result in higher sales?
- Are there unusual transactions that require further investigation?

Therefore, this project aims to analyze historical transaction data using Python and provide data-driven answers to these business questions.

5. DATASET DESCRIPTION

The dataset contains **one year of e-commerce transaction records**.

Each row represents an individual transaction/order record.

Dataset columns

Order_ID

Order_Date

Customer_ID

Product

Category

Quantity

Unit_Price

Discount

Sales

Region

Payment_Method

6. DATA DICTIONARY

This section is **very important** for professional documentation.

Column	Description	Data Type
Order_ID	Unique identifier for an order	String
Order_Date	Date on which the order was placed	Date
Customer_ID	Unique customer identifier	String
Product	Product purchased by the customer	String
Category	Product category	String
Quantity	Number of units purchased	Integer
Unit_Price	Price of one unit	Float
Discount	Discount applied to the transaction	Float
Sales	Total sales/revenue generated by the transaction	Float
Region	Geographic region of the customer/order	String
Payment_Method	Method used for payment	String

7. TOOLS & TECHNOLOGIES

Programming Language

Python libraries - Pandas, Numpy, Matplotlib

Pandas

Used for:

- Data loading
- Data cleaning
- Data transformation
- Grouping
- Aggregation
- Statistical analysis

NumPy

Used for:

- Numerical calculations
- Statistical operations
- Outlier detection
- Mathematical operations

Matplotlib

Used for:

- Bar charts
- Line charts
- Pie charts
- Scatter plots
- Data visualization

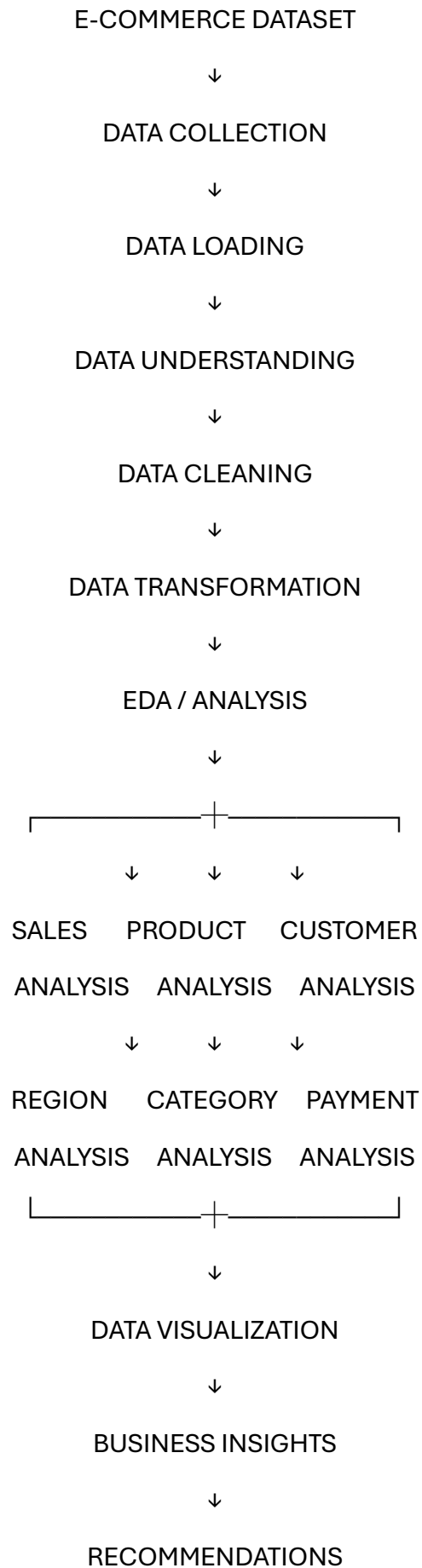
Development Environment

Visual Studio Code

Python Virtual Environment

8. PROJECT WORKFLOW

The project follows this workflow:



9. DATA LOADING

The dataset is loaded using Pandas.

Purpose

The purpose of this step is to import the raw dataset into a Pandas DataFrame for further analysis.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# We can change the index using index_col method
# data = pd.read_excel("sales_dataset.xlsx", index_col="Order_ID")
data = pd.read_excel("sales_dataset.xlsx")
```

10. DATA UNDERSTANDING

The dataset is examined using:

```
print(df.info())
print(df.describe())
# returns the number of rows and columns
print(df.shape)
print(df.head())
print(df.tail())
print(df.columns)
print(df.value_counts())
```

Output:

Info()

```
(env) PS C:\Users\perum\OneDrive\Desktop\pandas\Data_analytics_project> py sales_analysis.py
<class 'pandas.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Order_ID               500 non-null   str
1   Order_Date             500 non-null   str
2   Customer_ID            500 non-null   str
3   Product                500 non-null   str
4   Category               500 non-null   str
5   Quantity               500 non-null   int64
6   Unit_Price             500 non-null   float64
7   Discount               500 non-null   int64
8   Sales                  500 non-null   float64
9   Region                 500 non-null   str
10  Payment_Method          500 non-null   str
dtypes: float64(2), int64(2), str(7)
memory usage: 43.1 KB
```

describe()

	Quantity	Unit_Price	Discount	Sales
count	500.000000	500.000000	500.000000	500.000000
mean	3.942000	18997.051240	11.280000	66596.86682
std	1.976474	23944.852489	7.521619	106220.37642
min	1.000000	398.680000	0.000000	429.11000
25%	2.000000	3565.730000	5.000000	10546.83000
50%	4.000000	9180.880000	10.000000	28380.92000
75%	5.250000	22573.412500	15.000000	66813.07250
max	7.000000	116046.250000	25.000000	684744.92000

shape()

```
(500, 11)
```

head()

	Order_ID	Order_Date	Customer_ID	Product	Category	Quantity	Unit_Price	Discount	Sales	Region	Payment_Method
0	ORD1001	2025-01-01	CUST1034	Monitor	Computers	2	28754.70	15	48882.99	West	Net Banking
1	ORD1002	2025-01-01	CUST1045	Headphones	Accessories	1	10618.52	25	7963.89	South	Cash
2	ORD1003	2025-01-02	CUST1061	USB Drive	Storage	5	1852.54	10	8336.43	South	Debit Card
3	ORD1004	2025-01-03	CUST1083	Router	Networking	4	2907.16	15	9884.34	East	Credit Card
4	ORD1005	2025-01-03	CUST1123	Webcam	Accessories	2	1744.65	10	3140.37	South	Credit Card

Tail()

	Order_ID	Order_Date	Customer_ID	Product	Category	Quantity	Unit_Price	Discount	Sales	Region	Payment_Method
495	ORD1496	2025-12-28	CUST1113	USB Drive	Storage	2	3532.36	0	7064.72	East	Credit Card
496	ORD1497	2025-12-28	CUST1083	Power Bank	Accessories	4	889.33	15	3023.72	North	UPI
497	ORD1498	2025-12-29	CUST1043	Monitor	Computers	1	40363.96	25	30272.97	South	UPI
498	ORD1499	2025-12-30	CUST1116	Power Bank	Accessories	6	3021.54	15	15409.85	South	UPI
499	ORD1500	2025-12-31	CUST1078	Tablet	Mobiles	4	52258.05	20	167225.76	North	Cash

Columns()

```
Index(['Order_ID', 'Order_Date', 'Customer_ID', 'Product', 'Category',  
      'Quantity', 'Unit_Price', 'Discount', 'Sales', 'Region',  
      'Payment_Method'],  
      dtype='str')
```

value_counts() is used in Pandas to find **how many times each unique value appears in a column/Series**.

Order_ID	Order_Date	Customer_ID	Product	Category	Quantity	Unit_Price	Discount	Sales	Region	Payment_Method	
ORD1001	2025-01-01	CUST1034	Monitor	Computers	2	28754.70	15	48882.99	West	Net Banking	1
ORD1002	2025-01-01	CUST1045	Headphones	Accessories	1	10618.52	25	7963.89	South	Cash	1
ORD1003	2025-01-02	CUST1061	USB Drive	Storage	5	1852.54	10	8336.43	South	Debit Card	1
ORD1004	2025-01-03	CUST1083	Router	Networking	4	2907.16	15	9884.34	East	Credit Card	1
ORD1005	2025-01-03	CUST1123	Webcam	Accessories	2	1744.65	10	3140.37	South	Credit Card	1
..											
ORD1496	2025-12-28	CUST1113	USB Drive	Storage	2	3532.36	0	7064.72	East	Credit Card	1
ORD1497	2025-12-28	CUST1083	Power Bank	Accessories	4	889.33	15	3023.72	North	UPI	1
ORD1498	2025-12-29	CUST1043	Monitor	Computers	1	40363.96	25	30272.97	South	UPI	1
ORD1499	2025-12-30	CUST1116	Power Bank	Accessories	6	3021.54	15	15409.85	South	UPI	1
ORD1500	2025-12-31	CUST1078	Tablet	Mobiles	4	52258.05	20	167225.76	North	Cash	1
Name: count, Length: 500, dtype: int64											

This step helps determine:

- Number of records
- Number of columns
- Data types
- Numerical statistics
- Missing values
- Dataset structure

11. DATA CLEANING & PREPROCESSING

Before performing analysis, the dataset must be checked for quality issues.

Missing values

```
# Duplicate values
print("duplicates: ", df.duplicated().sum())

# Remove duplicates
df = df.drop_duplicates()
```

Output:

```
duplicates: 0
```

Convert date and Check data types

```
# convert date
df["Order_Date"] = pd.to_datetime(df["Order_Date"])

# check data types
print(df.dtypes)
```

Output

```
Order_ID          str
Order_Date      datetime64[us]
Customer_ID       str
Product          str
Category         str
Quantity         int64
Unit_Price       float64
Discount         int64
Sales           float64
Region           str
Payment_Method    str
dtype: object
```

12. KEY PERFORMANCE INDICATORS (KPIs)

The project calculates several important business KPIs.

KPI 1 — Total Revenue:

```
# total revenue
total_revenue = df["Sales"].sum()
# print("Total Revenue:", total_revenue)
```

Output:

```
Total Revenue: 33298433.409999996
```

Business meaning:

Total revenue generated during the analysis period.

KPI 2 — Total Orders:

```
# total orders
total_orders = df["Order_ID"].nunique()
print(f'Total orders: {total_orders}')
```

Output:

```
Total orders: 500
```

Business meaning:

Number of unique orders received.

KPI 3 — Average Order Value:

```
# Average order ValueError
average_order_value = total_revenue / total_orders
print(f'Average Order Value: {average_order_value}')
```

Output:

```
Average Order Value: 66596.86682
```

Business meaning:

Average revenue generated per order.

13. MONTHLY REVENUE ANALYSIS

Create a month field:

here .dt Access to work with datetime properties.

.to_period("M") converts each date to a monthly period (e.g.,2024-05), ignoring the exact day and time

when use it? it's great for grouping or aggregate data by a time span(month,quarter,year) without worrying about exact dates

Ex: 2024-05-15 and 2024-05-30 both become 2024-05

```
# Monthly Revenue Trends
# create a month column
df['Month'] = df['Order_Date'].dt.to_period("M")

monthly_sales = df.groupby("Month")["Sales"].sum()
print(monthly_sales)
```

Output:

```
Month
2025-01    2236031.54
2025-02    2012062.25
2025-03    1688144.59
2025-04    3524840.88
2025-05    2683113.19
2025-06    2668434.99
2025-07    4528444.87
2025-08    2021389.83
2025-09    3103148.88
2025-10    2730508.28
2025-11    3125245.00
2025-12    2977069.11
Freq: M, Name: Sales, dtype: float64
```

Visualization:

```
monthly_sales.plot(kind="line", marker = "o")

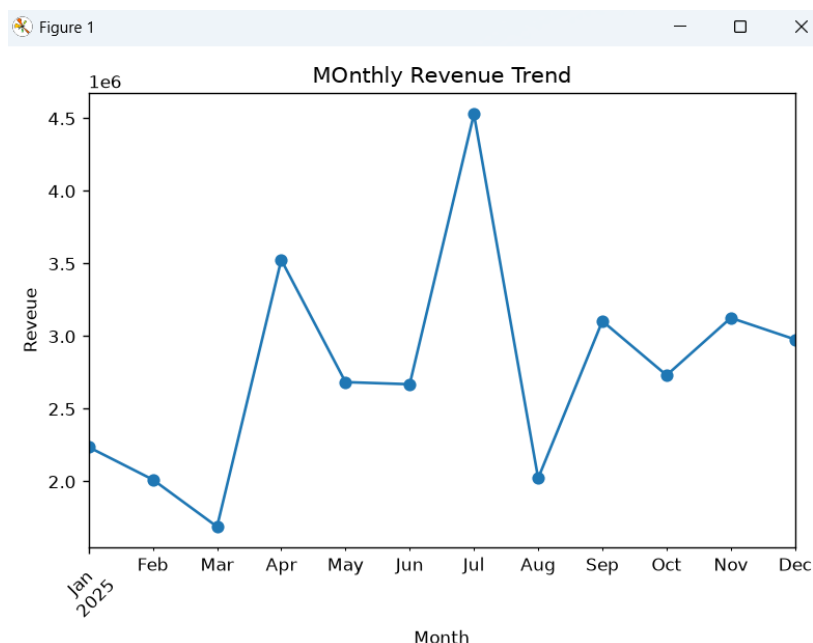
plt.title("Monthly Revenue Trend")
plt.xlabel("Month")
plt.ylabel("Revenue")
# This rotates all x-axis tick labels by 45 degrees, improving readability when labels overlap
plt.xticks(rotation = 45)
# it fixes cramped or overlapping plots by resizing and repositioning elements to fit neatly within the figure
plt.tight_layout()
plt.show()
```

Business Question

How does revenue change throughout the year?

To create a **line chart that shows the monthly revenue trend**. plt.figure(figsize=(12,5)) creates a wide chart, while plt.plot() plots the months on the X-axis and revenue on the Y-axis. monthly_sales.index.astype(str) represents the months, and monthly_sales.values represents the

revenue values. `marker="o"` adds circles to each data point. `plt.title()` gives the chart a title, `plt.xlabel()` and `plt.ylabel()` name the X-axis and Y-axis, `plt.xticks(rotation=45)` rotates the month names so they are easier to read, `plt.tight_layout()` adjusts the spacing, and `plt.show()` displays the final chart.



14. TOP 10 PRODUCTS

To find the **top 10 products based on total sales**, `df.groupby("Product")["Sales"]` groups the data according to each product and selects the Sales column, `.sum()` calculates the **total sales of each product**, `.sort_values(ascending=False)` sorts the products from **highest sales to lowest sales**, and `.head(10)` selects only the **top 10 products**. So, the final `top_products` variable contains the **10 best-selling products based on total sales**.

```
# Top 10 products
top_products = df.groupby("Product")["Sales"].sum().sort_values(ascending=False).head(10)

print(top_products)
```

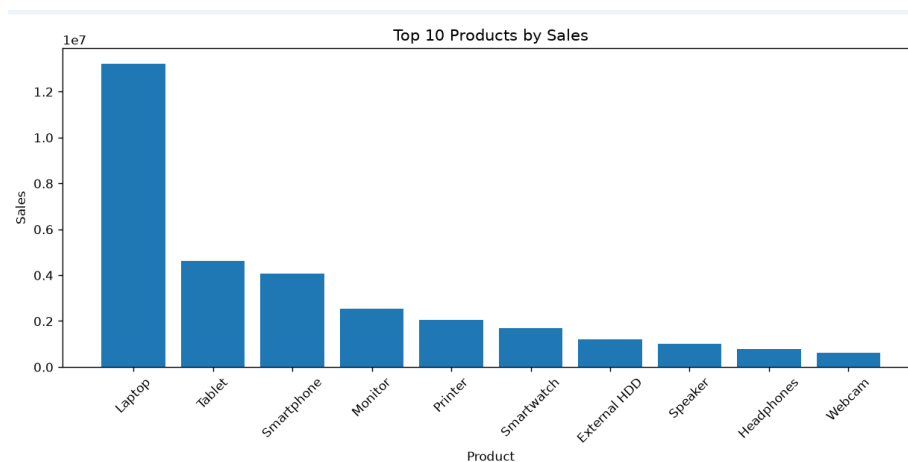
Output:

```
Product
Laptop      13218011.29
Tablet      4604080.50
Smartphone  4070500.66
Monitor     2543002.85
Printer     2033760.01
Smartwatch  1675981.32
External HDD 1196467.53
Speaker     997828.44
Headphones  786840.12
Webcam      605354.79
Name: Sales, dtype: float64
```

Visualization:

```
# Visualization
top_products.plot(kind="bar")
plt.figure(figsize=(10,5))
plt.bar(top_products.index.astype(str),top_products.values)
plt.title("Top 10 Products by Sales")
plt.xlabel("Product")
plt.ylabel("Sales")
plt.xticks(rotation = 45)
plt.tight_layout()
plt.show()
```

Output:



Business Question

Which products generate the highest revenue?

15. CATEGORY SALES ANALYSIS

To find the total sales for each product category, `df.groupby("Category")["Sales"]` groups the data based on each category and selects the Sales column, `.sum()` calculates the total sales of each category, and `.sort_values(ascending=False)` sorts the categories from highest sales to lowest sales. Finally, `print(category_sales)` displays the total sales of each category. This helps us identify which product category generates the most sales.

```
# sales by category
category_sales = df.groupby("Category")["Sales"].sum()
print(category_sales)
```

Output:

```
Category
Accessories    2188846.15
Audio           997828.44
Computers     17794774.15
Mobiles       8674581.16
Networking     575104.27
Storage       1391317.92
Wearables     1675981.32
Name: Sales, dtype: float64
```

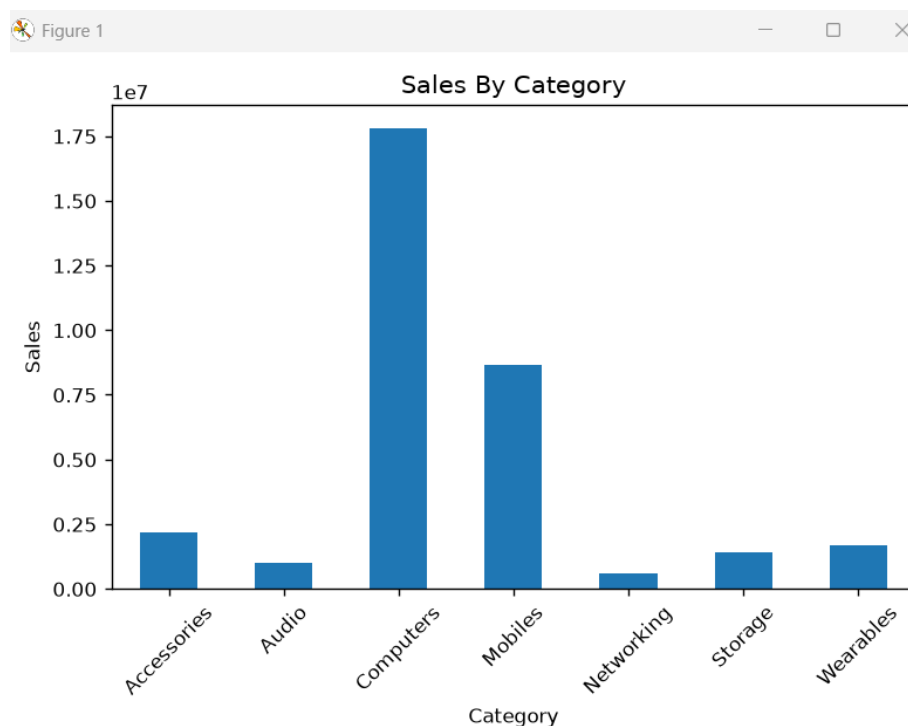
Visualization:

```
category_sales.plot(kind="bar")
plt.title("Sales By Category")
plt.xlabel("Category")
plt.ylabel("Sales")
plt.xticks(rotation = 45)
plt.tight_layout()
plt.show()
```

Business Question

Which product category contributes the most revenue?

Output:



16. REGIONAL SALES ANALYSIS

To find the total sales for each region, `df.groupby("Region")["Sales"]` groups the data based on each region and selects the Sales column, `.sum()` calculates the total sales of each region, and `.sort_values(ascending=False)` sorts the regions from highest sales to lowest sales. Finally, `print(region_sales)` displays the total sales for each region. This helps us identify which region generates the highest sales.

```
# sales by region
```

```
region_sales = df.groupby("Region")["Sales"].sum().sort_values(ascending=False)
print(region_sales)
```

Output:

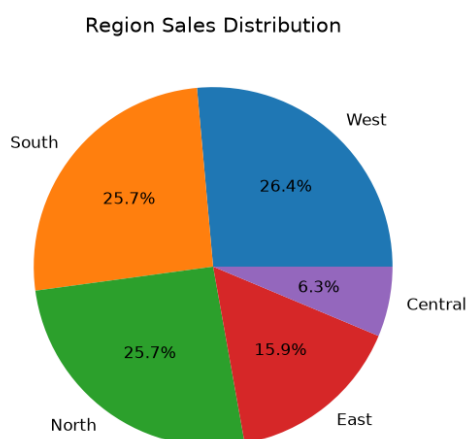
```
Region
West      8802759.13
South     8559862.54
North     8550274.03
East      5283825.83
Central   2101711.88
Name: Sales, dtype: float64
```

This helps identify the strongest and weakest geographic markets.

Visualization:

```
.....
region_sales.plot(kind = "pie",autopct = "%1.1f%%")
plt.title("Region Sales Distribution")
plt.show()
```

Output:



17. PAYMENT METHOD ANALYSIS

To find how many orders were made using each payment method.

`df["Payment_Method"]` selects the Payment Method column, and `.value_counts()` counts how many times each payment method appears in the dataset. Finally, `print(payment_counts)` displays count of each payment method is most commonly used by customers.

```
payment_count = df["Payment_Method"].value_counts()

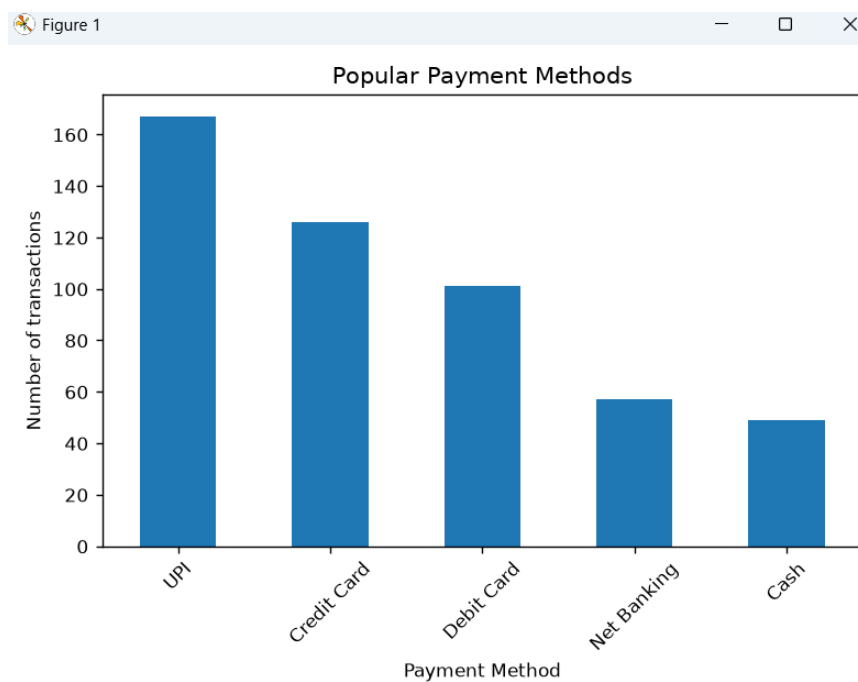
print(payment_count)
```

Output:

```
Payment_Method
UPI          167
Credit Card 126
Debit Card   101
Net Banking   57
Cash         49
Name: count, dtype: int64
```

Visualization:

```
payment_count.plot(kind="bar")
plt.title("Popular Payment Methods")
plt.xlabel("Payment Method")
plt.ylabel("Number of transactions")
plt.xticks(rotation = 45)
plt.tight_layout()
plt.show()
```



Business Question

Which payment method is most frequently used by customers?

18. HIGHEST & LOWEST SALES MONTH

Highest

```
# highest and lowest sales month
# idxmax() returns the index/label where the maximum occurs.
highest_month = monthly_sales.idxmax()
highest_sales = monthly_sales.max()
print(f'Highest sales Month: {highest_month}')
print(f'Highest sales: {highest_sales}')
```

Output:

```
Highest sales Month: 2025-07
Highest sales: 4528444.87
```

Lowest

```
lowest_month = monthly_sales.idxmin()
lowest_sales = monthly_sales.min()

print(f'Lowest sales Month: {lowest_month}')
print(f'Lowest sales: {lowest_sales}')
```

Output:

```
Lowest sales Month: 2025-03
Lowest sales: 1688144.59
```

19. DISCOUNT VS SALES ANALYSIS

Calculate correlation:

```
correlation = df["Discount"].corr(df["Sales"])
print(f'Discount-sales Correlation: {correlation}')
```

Output:

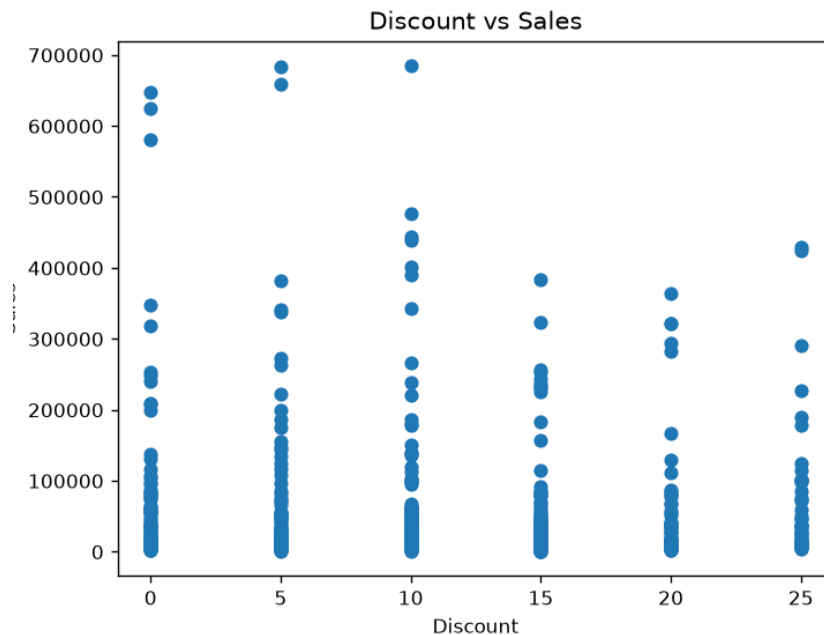
```
Discount-sales Correlation: -0.07765533531900423
```

Visualization:

```
# visualization
plt.scatter(df["Discount"],df["Sales"])

plt.title("Discount vs Sales")
plt.xlabel("Discount")
plt.ylabel("Sales")
plt.show()
```

Output:



Business Question

Is there a relationship between the discount offered and sales generated?

scatter plot to analyze the relationship between Discount and Sales**. `plt.scatter(df["Discount"], df["Sales"])` places **Discount on the X-axis** and **Sales on the Y-axis**, where each dot represents one record/order in the dataset. `plt.title("Discount vs Sales")` gives the graph a title, while `plt.xlabel("Discount")` and `plt.ylabel("Sales")` label the axes. `plt.show()` displays the graph. The main purpose of this graph is to check whether **changes in discount are related to changes in sales**. If the points generally move upward, higher discounts may be associated with higher sales; if there is no clear pattern, there may be **little or no relationship** between discount and sales.

Important analytical note

A correlation does **not automatically mean causation**.

For example:

High discount



High sales

doesn't necessarily prove that the discount caused the high sales.

20. OUTLIER / ANOMALY ANALYSIS

We can identify unusually high or low transactions using the **IQR method**.

```
sales = df["Sales"]

mean_sales = np.mean(sales)
median_sales = np.median(sales)
std_sales = np.std(sales)

# Q25 → 25% of sales values are below this value
# Q50 → 50% of sales values are below this value
# Q75 → 75% of sales values are below this value

q25 = np.percentile(sales, 25)
q50 = np.percentile(sales, 50)
q75 = np.percentile(sales, 75)

print(f"Mean: {mean_sales}")
print(f"Median: {median_sales}")
print(f"Standard Deviation: {std_sales}")
print(f"25th Percentile: {q25}")
print(f"50th Percentile: {q50}")
print(f"75th Percentile: {q75}")
```

Output:

```
Mean: 66596.86682
Median: 28380.92
Standard Deviation: 106114.1028806463
25th Percentile: 10546.83
50th Percentile: 28380.92
75th Percentile: 66813.07250000001
```

```

# IQR (Interquartile Range) method is used to identify the Outliers
# An outlier is a data value that is significantly different from the majority of observations in a dataset.
# Outlier analysis helps identify unusual transactions, possible data errors,
# and exceptional business events that may affect statistical analysis.
# IQR = middle 50% of your Sales data spread.
IQR = q75 - q25
lower_bound = q25 - 1.5 * IQR
upper_bound = q75 + 1.5 * IQR

outliers = df[(sales < lower_bound) | (sales > upper_bound)]

print(f"Lower Bound: {lower_bound}")
print(f"Upper Bound: {upper_bound}")
print(f"Number of Outliers: {len(outliers)}")

```

Output:

```

Lower Bound: -73852.53375000002
Upper Bound: 151212.43625000003
Number of Outliers: 59

```

Business Question

Are there transactions with unusually high or low sales values?

These transactions may require further investigation.

21. DATA VISUALIZATIONS

The project includes different visualizations to present the sales performance, product performance, category performance, regional performance, payment preferences, and relationship between discounts and sales in an easy-to-understand graphical format.

- Line Chart – Monthly Revenue Trends: Shows how revenue changes month by month and helps identify growth patterns, declining periods, and seasonal trends.
- Bar Chart – Top 10 Products: Displays the 10 highest-selling products, helping identify the products that contribute most to overall sales.
- Bar Chart – Category Sales: Shows the total sales generated by each product category, making it easy to compare category performance.
- Bar Chart – Regional Sales: Compares the total sales across different regions and helps identify the strongest and weakest-performing regions.
- Pie Chart – Payment Distribution: Shows the proportion of orders made using different payment methods, helping understand customer payment preferences.
- Scatter Plot – Discount vs Sales: Shows the relationship between discount percentage and sales. It helps determine whether higher discounts are associated with higher sales and identify any noticeable patterns or outliers.

22. KEY BUSINESS INSIGHTS

This section should **not be written before running the analysis**.

After running your Python code, document findings like:

1. The company generated **₹33298433.40** total revenue during the year.
2. The company received **500** unique orders.
3. The average order value was **₹ 66596.86**.
4. **July** recorded the highest monthly revenue.
5. Product **Laptop** was the highest-revenue product.
6. Category **computers** contributed the largest share of revenue.
7. Region **West** generated the highest sales.
8. **UPI** was the most frequently used payment method.
9. A positive/negative/weak relationship was observed between **discounts** and **sales**.
10. **59** transactions were identified as statistical outliers.

23. BUSINESS RECOMMENDATIONS

Based on the findings, recommendations can be provided.

For example:

1. Product Strategy

Focus inventory and marketing efforts on high-performing products that generate the most sales and revenue. Maintain sufficient stock of popular products to avoid stock-outs and lost sales. At the same time, analyze low-performing products and consider improving, promoting, or discontinuing them based on their performance.

2. Regional Strategy

Identify regions that generate high sales or show strong growth potential and increase marketing efforts in those areas. Businesses can provide region-specific offers, promotions, and product recommendations to attract more customers. This can help improve sales and expand the customer base in promising regions.

3. Discount Strategy

Evaluate discount campaigns based on their actual impact on sales and revenue, rather than simply offering larger discounts. Analyze whether discounts increase sales enough to compensate for the reduction in selling price. Focus on targeted discounts for specific products, customers, or periods where they are most effective.

4. Payment Strategy

Optimize the **most frequently used payment methods** and ensure they are fast, secure, and reliable. If customers strongly prefer a particular payment method, the business should make sure it is always available and easy to use. Providing multiple convenient payment options can also reduce payment failures and improve the overall customer experience.

5. Seasonal Strategy

Analyze historical sales data to identify **months or periods with higher sales**. Prepare sufficient inventory, staff, promotions, and marketing campaigns before these high-demand periods. Seasonal offers and targeted advertising can help the business take advantage of increased customer demand and maximize revenue.

24. CONCLUSION

The E-Commerce Sales Data Analysis project demonstrates how Python can be used to transform raw transaction data into meaningful business insights.

Using Pandas, NumPy, and Matplotlib, the project performs data cleaning, exploratory data analysis, statistical analysis, visualization, and business interpretation.

The analysis provides insights into:

- Revenue performance
- Order performance
- Product performance
- Category performance
- Regional performance
- Customer payment preferences
- Monthly trends
- Discount relationships
- Transaction anomalies

The project demonstrates a complete **Python-based Data Analytics workflow**